

AI-Based Food Recognition

with Nutrition-Aware Recommendations



Presented and made by

Jaspreet Singh (2022BCSE019)

Inderpal Singh (2022BCSE037)

Guide: Dr. Lavanya Madhuri Bollipo

Co-Guide: Dr. Insha Majeed

Table of Contents

01 Title Slide

Project introduction and basic details.

02 Table of Contents

Overview of the presentation structure.

03 Introduction

Context and core goals of the AI system.

04 Problem Statement

Key challenges in food recognition.

05–06 Literature Review

Analysis of existing research and methods.

07 Objectives of the Study

Primary and secondary research goals.

08 Proposed System Workflow

Step-by-step logic of the AI architecture.

09–11 Datasets

Training data and nutrition databases used.

13–15 Implementation

Technical setup and coding details.

16–17 System Pipelines

Backend processing and API flow.

18–19 UI & Outputs

User interface design and results.

20 Project Demo

21 Conclusion

Introduction



Growing Need

Food image analysis is an emerging application of artificial intelligence in health and dietary support systems.



This Project

The system combines **YOLO-based detection** with a nutrition database to identify items from uploaded meal images.



Core Goal

Implemented as a **web-based platform**, this solution offers a practical and accessible tool for nutritional analysis.

Project Scope: An AI-based food recognition system specifically designed for the visual complexity of Indian cuisine.

Problem Statement



Health Tracking Importance

Food intake tracking is vital for dietary awareness and health monitoring.



Inefficiency of Manual Logs

Traditional logging is time-consuming and impractical for everyday use.



Recognition Complexity

Indian cuisine has high visual similarity, making AI detection challenging.



Conclusion: An AI-based system is required to automate food detection and provide real-time nutritional mapping from images.

Literature Review: Systems & Gaps

PAPER	IMPLEMENTATION	LIMITATION / GAP	OUR SOLUTION
Khana: Comprehensive Indian Cuisine Dataset (2025)	Created a large Indian cuisine dataset with 131K images across 80 labels and trained baseline models.	Focused on image-level understanding; doesn't solve multi-item detection in cluttered plates.	Uses YOLO11s to identify multiple food items in one image with bounding boxes.
What is there in an Indian Thali? (2025)	Built a pipeline for segmentation and weight estimation using a dedicated scanner setup.	Depends on specialized hardware kiosk; impractical for normal web or mobile users.	Scalable web-based solution works from a single uploaded image without special hardware.
Enhancing FKG.in: Indian Food Composition (2024)	Developed an automated workflow using knowledge graphs and LLMs from Indian recipe text.	Pipeline is text-dependent and cannot analyze a meal directly from an image.	Implements a fully visual pipeline where detection and nutritional mapping happen automatically.

Literature Review: Implementation

Reference Paper	Main Contribution	Relation to Our Work
Indian Food Composition Database (2024)	Developed the open-access Indian Nutrient Databank (INDB) with 1,095 items and 1,014 common recipes.	Directly used for the nutritional backend to map detected items to records.
IndianFoodNet: Detecting Indian Food (2023)	Introduced detection dataset with 5,500+ images and evaluated YOLO-based detectors.	Foundation for our object detection , extended via YOLO11s.
NutriGen: Personalized Meal Generator (2025)	Proposed LLM-based framework for meal-plan generation based on calorie goals.	Reference for the recommendation phase , focusing on LLM nutrition.
AI-Driven Personalized Meal Planning (2025)	Web-based platform for culturally aware personalized meal recommendations.	Reference for culturally tailored planning and guidance.

Objectives of the Study

01

Indian Food Recognition

Recognize Indian food items from user-provided images using an AI-based object detection model.

02

Nutrition Mapping

Map detected food items to a nutrition database and display macronutrient details (calories, protein, fat, carbs).

03

Multi-Item Support

Support simultaneous detection of multiple food items within a single image.

04

Web Interface

Provide a simple drag-and-drop web interface accessible to the users.

Semantic Workflow of the Proposed System



Step 1

Image Upload

User uploads food photo via drag-and-drop web interface



Step 2

Food Detection

YOLO11s model detects and labels all food items



Step 3

DB Matching

Detected names matched with INDB database



Step 4

Result Display

UI renders bounding boxes and nutrition panel



Backend Layer

FastAPI server handling inference logic and DB lookups



Frontend Layer

Next.js application providing responsive user experience

Dataset for Model Training

Source: IndianFoodNet image dataset from roboflow



Dataset Structure

In YOLO format with distinct train, validation, and test splits for supervised learning.

Target Classes

Configuration specifies 30 target food classes, including Biryani, Dosa, Samosa, Dal, and Palak Paneer.

Volume & Density

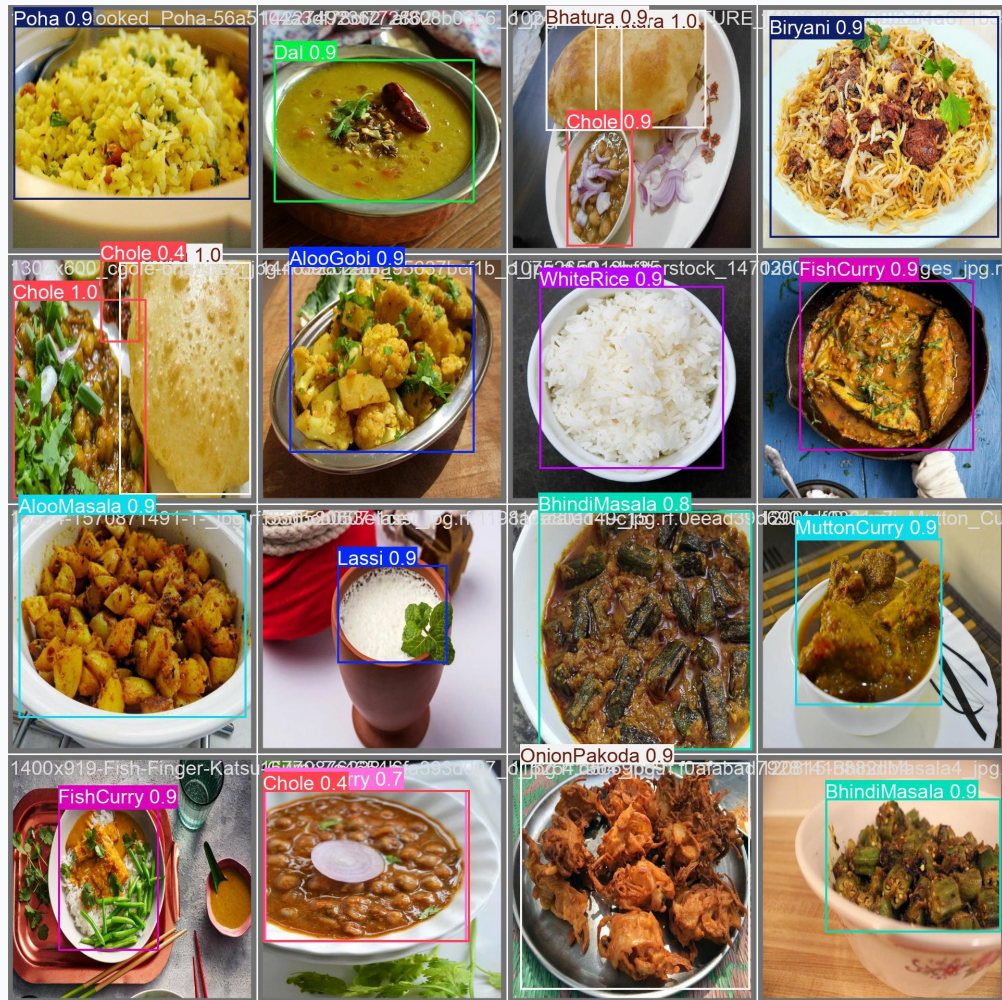
- 11,387 training images • 1,073 validation images • ~16k annotations

Core Utility

Primary source for training the YOLO11s food detection model architecture.

```
! datayaml X
data > indianfoodnet_yolo > ! datayaml

1 names:
2 - AlooGobi
3 - AlooMasala
4 - Bhatara
5 - BhindiMasala
6 - Biryani
7 - Chai
8 - Chole
9 - CoconutChutney
10 - Dal
11 - Dosa
12 - DumAloo
13 - FishCurry
14 - Ghevar
15 - GreenChutney
16 - GulabJamun
17 - Idli
18 - Jalebi
19 - Kebab
20 - Kheer
21 - Kulfi
22 - Lassi
23 - MuttonCurry
24 - OnionPakoda
25 - PalakPaneer
26 - Poha
27 - RajmaCurry
28 - RasMalai
29 - Samosa
30 - ShahiPaneer
31 - WhiteRice
32 nc: 30
33 path: gx/Projects/Final_Year_Project/data/Indianfoodnet_yolo
34 roboflow:
35 license: CC BY 4.0
36 project: indianfoodnet
37 url: https://universe.roboflow.com/indianfoodnet/indianfoodnet/dataset/1
38 version: 1
39 workspace: indianfoodnet
40 test: ../test/images
41 train: ../train/images
42 val: ../valid/images
```



Nutrition Dataset and Database

	yolo...	matched_food_name	match_score
	Filter...	Filter...	Filter...
1	AlooGobi	Cauliflower kheer (Phoolgobhi ki kheer)	0.6875
2	AlooMasala	Dry potato (Sookhe aloo)	0.6666666666666666
3	Bhatura	Bhatura	1
4	Chai	Hot tea (Garam Chai)	0.5
5	Chole	Chickpeas curry (Safed channa curry)	0.5714285714285714
6	CoconutChutney	Coconut chutney (Nariyal ki chutney)	1
7	Dal	Lentil soup	0.6
8	DumAloo	Dum aloo	1
9	FishCurry	Fish curry (Machli curry)	1
10	GreenChutney	Green chutney	1
11	GulabJamun	Gulab Jamun with khoya	0.5263157894736842
12	Idli	Idli	1
13	Kebab	Boti kebab	0.5555555555555556
14	Kheer	Rice kheer (Chawal ki kheer)	0.5555555555555556
15	Kulfi	Kulfi	1
16	Lassi	Lassi (salted)	1
17	MuttonCurry	Mutton chops	0.5454545454545454
18	OnionPakoda	Onion pakora/pakoda (Pyaz ke pakode)	0.6470588235294118
19	PalakPaneer	Cottage cheese pie	0.8125
20	Poha	Poha	1
21	RajmaCurry	Kidney bean curry (Rajmah curry)	1
22	RasMalai	Rasmalai	1
23	Samosa	Potato samosa (Aloo ka samosa)	0.5
24	ShahiPaneer	Shahi paneer	1
25	WhiteRice	Curd rice (Dahi bhaat/Dahi chawal/ Perugu annam/Dad...	0.5

Comprehensive Database

Maintains food names, category labels, and macronutrient values (calories, protein, carbs, fat, fiber).

Standardized Retrieval

Maps detected items to standardized records for per-100g nutritional retrieval.

Precise Mapping

Includes a `yolo_mappings` table for high-accuracy label-to-record matching.

Rows: 1,005								Filter 1,005 rows...
	id Filter...	name Filter...	normalized_name Filter...	calories Filter...	protein_g Filter...	carbs_g Filter...	fat_g Filter...	
109	109	Mutton biryani/biriyani	muttonbiryanibiriyani	190.76315307617188	7.376446723937988	22.495088577270508	7.724486827850342	
110	110	Vegetable biryani/biriyani	vegetablebiryanibiriyani	174.60516357421875	3.1605417728424072	18.557273864746094	9.507964134216309	
111	111	Lemon rice (Pulihora, Elumichai sadam, Chitranna)	lemonrice	176.30288696289062	4.256220817565918	21.622270584106445	7.8798065185546875	
112	112	Sweet rice (Meethe chawal)	sweetrice	215.41348266601562	2.095991611480713	42.59217071533203	4.565640926361084	
113	113	Curd rice (Dahi bhaat/Dahi chawal/ Perugu annam/Dad...	curdrice	195.73703002929688	5.753233909606934	32.93044662475586	4.316627025604248	
114	114	Tamarind rice (Chintapandu pulihora/Puliyodharai/Puli ...	tamarindrice	373.0386047363281	7.453181743621826	65.08139038085938	8.532773971557617	
115	115	Spanish rice	spanishrice	164.24085998535156	4.189563274383545	31.09807014465332	2.2327356338500977	
116	116	Chinese fried rice	chinesefriedrice	120.58412170410156	4.05177116394043	13.401684761047363	5.434627056121826	
117	117	Macroni cheese pie	macronicheesepie	170.62753295898438	5.85098123550415	21.09503173828125	7.483591079711914	
118	118	Vegetable chowmein	vegetablechowmein	129.71388244628906	3.066948175430298	15.444334983825684	6.358810901641846	
119	119	Chicken chowmein	chickenchowmein	151.3335723876953	6.331267833709717	13.301461219787598	8.234783172607422	
120	120	Cheese noodle ring	cheesenoodlering	133.66921997070312	6.224301815032959	13.955496788024902	6.117282867431641	
121	121	Spaghetti and cheese balls in tomato sauce	spaghettiandcheeseballsintomato...	508.322021484375	2.4087061882019043	7.815141677856445	52.08030700683594	
122	122	Penne platter	penneplatter	177.40061950683594	5.040764808654785	20.872121810913086	8.709636688232422	
123	123	Pasta hot pot	pastahotpot	125.17289733886719	5.837541580200195	11.601784706115723	6.466379642486572	
124	124	Chicken lasagne	chickenlasagne	187.4915313720703	10.452420234680176	13.902274131774902	10.36139965057373	
125	125	Fettuccine with spinach sauce	fettuccinewithspinachsauce	129.20672607421875	3.103574752807617	12.18928050994873	7.828339576721191	
126	126	Naan	naan	286.44964599609375	8.05210018157959	51.75419998168945	4.987837314605713	
127	127	Bhatura	bhatura	793.2029418945312	1.6303993463516235	10.733963012695312	82.5638427734375	
128	128	Idli	idli	137.53526306152344	4.637406349182129	28.17755699157715	0.33316707611083984	
129	129	Masala dosa	masaladosa	164.5846710205078	3.2877273559570312	19.566587448120117	7.840357303619385	
130	130	Semolina dosa (Suji/Rava dosa)	semolinadosa	227.05209350585938	7.300220012664795	32.84082794189453	7.116666316986084	
131	131	Onion tomato uttapam	oniontomatouttapam	462.0191955566406	1.9062505960464478	11.315990447998047	45.30297088623047	
132	132	Paneer kaathi roll	paneerkaathiroll	285.942138671875	6.916443347930908	20.103469848632812	19.554462432861328	
133	133	Makki ki roti	makkikiroti	263.9683532714844	3.4924707412719727	24.191999435424805	16.849411010742188	
134	134	Washed moong dal (Dhuli moong ki dal)	washedmoongdal	50.00269317626953	2.677743673324585	5.906931400299072	1.680992841720581	
135	135	Washed urad dal (Dhuli urad ki dal)	washeduraddal	61.32363510131836	2.5109291076660156	5.8411149978637695	2.9800169467926025	
+	1,006	Split bengal gram with bottle gourd (Channa dal with g...	splitbengalgramwithbottlegourd	73.25393676757812	2.789580821990967	7.101042747497559	3.602670431137085	

Tools & Technologies Used



Frontend

Next.js: React framework for user interface.

TypeScript: Statically typed language for reliability.

Tailwind CSS: Utility-first CSS for responsive design.



Backend

Python: Core logic and service integration.

FastAPI: High-performance RESTful API services.

UVicorn: ASGI server for backend application.



AI / ML

YOLOv11s & Ultralytics: Food recognition model & pipeline.

PyTorch: Framework for training and runtime inference.

CUDA: GPU platform for performance acceleration.



Supporting Components

SQLite &aiosqlite: Lightweight async database system.

Pillow: Imaging library for loading and preprocessing.

OpenCV: CV library for auxiliary image operations.

Model Training Configuration

```
# Cell 2: YOLO11s detection training
# DATA_YAML and DEVICE are set in Cell 0/1 - run those cells first
print('=== Starting YOLO11s Training ===')
print(f'Dataset : {DATA_YAML}')
print(f'Device   : {DEVICE} | Batch : {BATCH} | Workers: {WORKERS} | Cloud: {CLOUD}')

model = YOLO('yolo11s.pt')

training_args = {
    'data'       : DATA_YAML, # resolved in Cell 0 - works on Kaggle + local
    'task'       : 'detect',
    'epochs'     : 100,
    'imgsz'      : 640,
    'batch'      : BATCH,      # 16 on Kaggle P100/T4, 4 on local 4GB
    'patience'  : 20,
    'device'     : DEVICE,     # 0 (GPU) or 'cpu' (fallback)
    'workers'    : WORKERS,
    'amp'        : True,
    'save_period': 10,
    'project'    : 'models/runs',
    'name'       : 'yolo11s_indian',
}

print(f'Training args: {training_args}')
results = model.train(**training_args)
print('\nYOLO11s training completed!')
```

Model & Task

Utilizing YOLO11s.pt for object detection tasks with optimized weights.

Infrastructure

Configured for Kaggle P100/T4 GPUs with a batch size of 16 to maximize throughput.

Optimization

Trained for 100 epochs with early stopping (patience: 20) to prevent overfitting.

Image Resolution

Input images scaled to 640×640 for a balanced trade-off between speed and accuracy.

Training Outputs

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
90/100	4.98G	0.4298	0.3217	1.008	31	640: 100% ————— 712/712 3.0it/s 3:56<0.3s
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ————— 34/34 3.5it/s 9.8s<0.3s
	all	1073	1654	0.907	0.879	0.933 0.832

Closing dataloader mosaic

augmentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', n

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
91/100	4.98G	0.3707	0.2126	0.9697	20	640: 100% ————— 712/712 3.0it/s 3:55<0.3s
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ————— 34/34 3.5it/s 9.7s<0.3s
	all	1073	1654	0.91	0.876	0.932 0.832

EarlyStopping: Training stopped early as no improvement observed in last 20 epochs. Best results observed at epoch 71, best model s
To update EarlyStopping(patience=20) pass a new patience value, i.e. `patience=300` or use `patience=0` to disable EarlyStopping.

91 epochs completed in 6.204 hours.

Optimizer stripped from /kaggle/working/runs/detect/models/runs/yolo11s_indian/weights/last.pt, 19.2MB

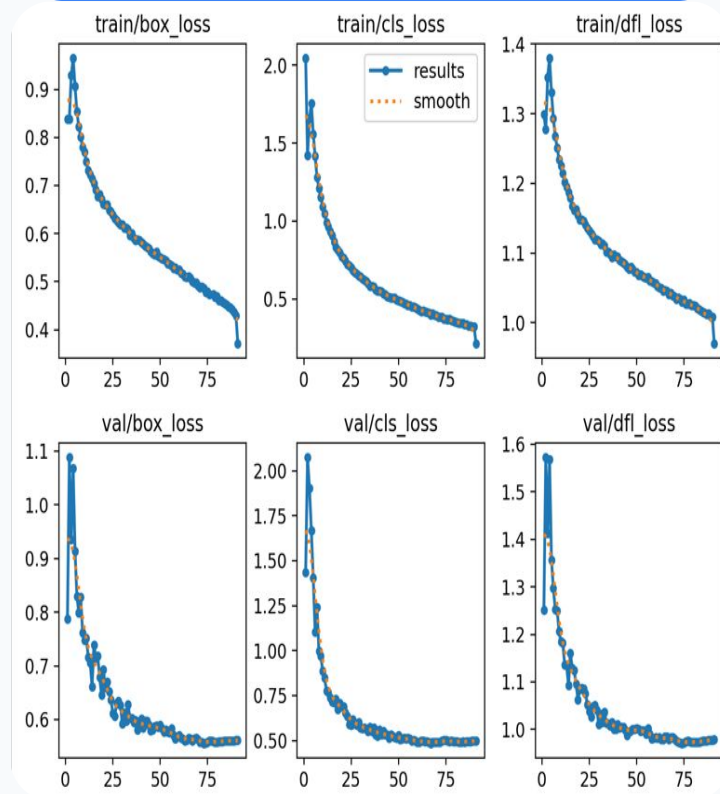
Optimizer stripped from /kaggle/working/runs/detect/models/runs/yolo11s_indian/weights/best.pt, 19.2MB

Validating /kaggle/working/runs/detect/models/runs/yolo11s_indian/weights/best.pt...

Ultralytics 8.4.33 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

YOLO11s summary (fused): 101 layers, 9,424,410 parameters, 0 gradients, 21.4 GFLOPs

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% ————— 34/34 3.1it/s 10.8s<0.3s
all	1073	1654	0.908	0.877	0.934	0.833
AlooGobi	39	40	0.861	0.825	0.919	0.846
AlooMasala	39	39	0.878	0.925	0.936	0.902
Bhatuna	44	87	0.917	0.888	0.958	0.848
BhindiMasala	43	45	0.958	0.911	0.974	0.913
Biryani	42	45	0.921	0.956	0.971	0.912
Chai	39	60	0.874	0.923	0.947	0.741



Food Detection Pipeline

- **Dynamic Loading:** The trained YOLO11s model is loaded dynamically from the configured path to enable runtime food detection.
- **Format Conversion:** The uploaded image is converted into RGB format and passed to the model for inference.
- **Methodological Alignment:** Detection uses confidence and IoU thresholds consistent with training, ensuring alignment between training and deployment.
- **Structured Output:** Generates detection data containing predicted food labels, confidence scores, and bounding-box coordinates.

```
82         return None
83
84     # Filter by per-class minimum confidence, then pick highest
85     valid_boxes = []
86     for box in detections:
87         class_id = int(box.cls[0])
88         class_name = self.model.names[class_id]
89         confidence = float(box.conf[0])
90         min_conf = WEAK_CLASS_MIN_CONF.get(class_name, conf)
91         if confidence >= min_conf:
92             valid_boxes.append((box, class_name, confidence))
93         else:
94             logger.info(f"Skipping {class_name} at {confidence:.3f} (min={min_conf})")
95
96     if not valid_boxes:
97         logger.warning("All detections below per-class minimum confidence")
98         return None
99
100     # Return ALL valid detections (sorted by confidence)
101     valid_boxes.sort(key=lambda x: x[2], reverse=True)
102
103     detections = []
104     for box, class_name, confidence in valid_boxes:
105         bbox = box.xyxy[0].cpu().numpy().astype(int).tolist()
106         detections.append({
107             "food_label" : class_name,
108             "confidence" : round(confidence, 4),
109             "bounding_box" : bbox
110         })
111
112     logger.info(f"Detected {len(detections)} foods: {[d['food_label'] for d in detections]}")
113
114     return {
115         "detections" : detections,
116         "img_width" : img.width,
117         "img_height" : img.height
118     }
```


Nutrition Mapping and Retrieval Pipeline

```
141 # Fuzzy matching
142 db_names = list(self._cached_foods.keys())
143 matched_name, score = find_best_match(food_label, db_names)
144
145 if matched_name and score >= 0.4: # Lower threshold for last resort
146     logger.info(f"Fuzzy match: {food_label} → {matched_name} (score: {score:.2f})")
147     async with aiosqlite.connect(str(self.db_path)) as db:
148         return await self._get_nutrition_by_name(db, matched_name, food_label)
149
150 logger.warning(f"No match found for '{food_label}'")
151 return None
152
153 except Exception as e:
154     logger.error(f"Error getting nutrition for {food_label}: {e}")
155     raise
156
157 async def _get_nutrition_by_name(self, db, db_food_name: str, original_label: str) -> dict:
158     """Get nutrition data by exact database food name"""
159     cursor = await db.execute(
160         "SELECT name, calories, protein_g, carbs_g, fat_g FROM indb_foods WHERE name = ?",
161         (db_food_name,)
162     )
163     row = await cursor.fetchone()
164
165     if row:
166         name, calories, protein, carbs, fat = row
167         return {
168             "food_label": original_label,
169             "display_name": name, # Return the ACTUAL database food name
170             "found": True,
171             "macros": {
172                 "calories": float(calories),
173                 "protein_g": float(protein),
174                 "carbs_g": float(carbs),
175                 "fat_g": float(fat)
176             },
177             "macros_unit": "per_100g",
178             "nutrition_source": "INDB",
179             "food_not_found": False
180         }
181     return None
```

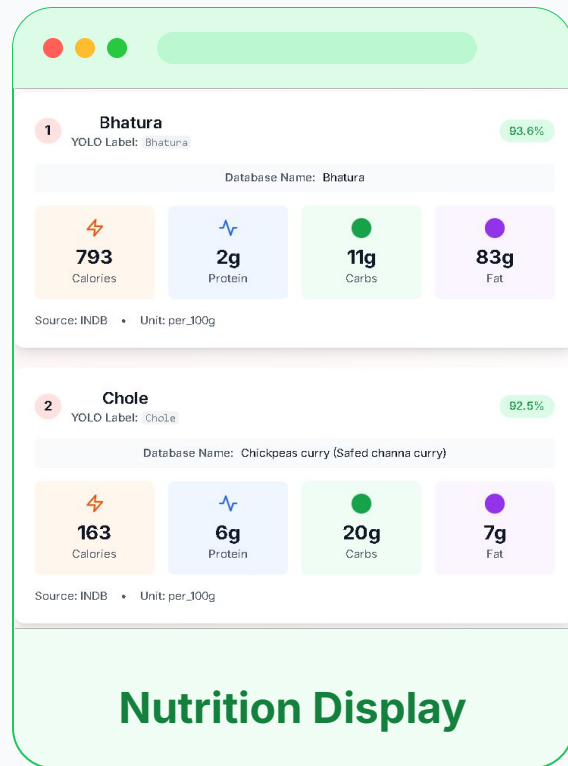
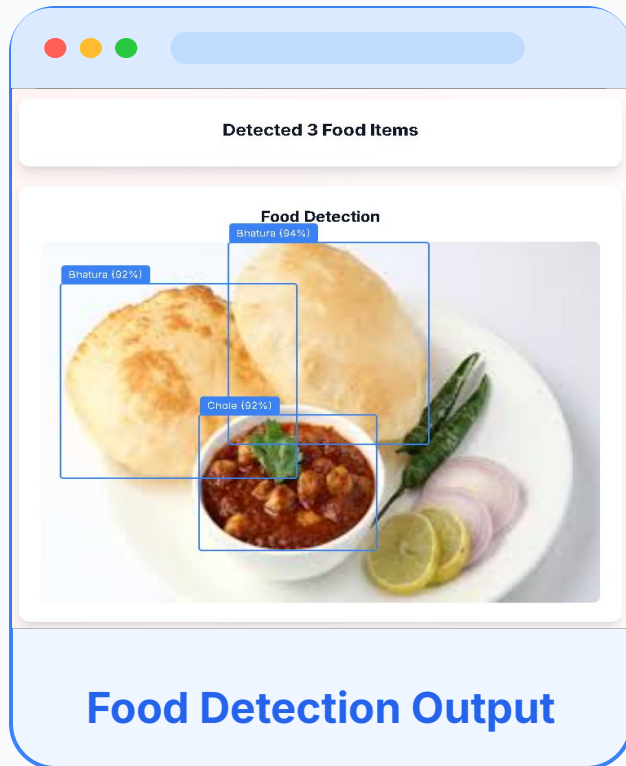
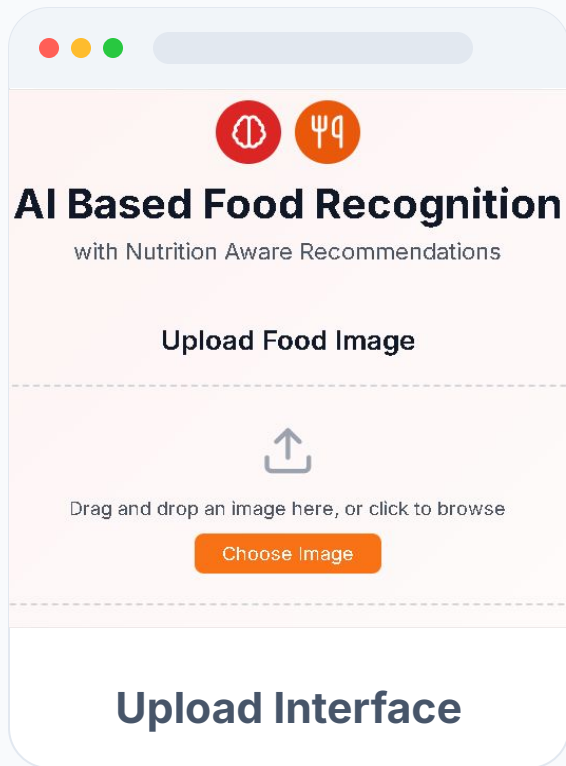
- **Database Connection:** The nutrition module connects to the SQLite nutrition.db database and prepares food names for fast lookup.
- **Fuzzy Matching:** It matches detected food labels with database entries using exact and fuzzy matching methods.
- **Robust Lookup:** This helps the system handle cases where the detected name and database name are not exactly the same.
- **Data Retrieval:** After a match is found, the system retrieves calories, protein, carbohydrates, and fat values.
- **Structured Output:** The output is returned in a structured per-100g format using INDB-based nutrition data.

User Interface

- **Next.js Implementation:** The frontend is implemented as a Next.js main page that manages the user interaction flow for food analysis.
- **Analysis Workflow:** It captures the uploaded image, sends it to the backend analysis endpoint, and waits for the detection response.
- **Data Presentation:** The returned prediction and nutrition data are then passed to UI components for display.

```
3 import React, { useState, useCallback } from 'react'
4 import ImageUpload from '@/components/ImageUpload'
5 import NutritionLabel from '@/components/NutritionLabel'
6 import ResultsPanel from '@/components/ResultsPanel'
7 import { Brain, Utensils } from 'lucide-react'
8
9 You, last week | 1 author (You)
10 interface FoodDetection {
11   food_label: string
12   display_name: string
13   confidence: number
14   bounding_box: [number, number, number, number]
15   macros: {
16     calories: number
17     protein_g: number
18     carbs_g: number
19     fat_g: number
20   } | null
21   macros_unit: string
22   nutrition_source: string | null
23   nutrition_not_found?: boolean
24 }
25
26 You, last week | 1 author (You)
27 interface FoodAnalysis {
28   detections: FoodDetection[]
29   img_width: number | null
30   img_height: number | null
31   food_not_found: boolean
32   message?: string
33 }
```

Project Screenshots



Project Demo



AI Based Food Recognition

with Nutrition Aware Recommendations

Upload Food Image



Drag and drop an image here, or click to browse

Choose Image

Conclusion

- ✓ A working system for real-time Indian food identification from images.
- ✓ Integrated nutrition mapping using an Indian food nutrition database.
- ✓ An accessible, web-based interface combining computer vision and nutritional intelligence in one pipeline.

Addresses the research gaps identified from current literature.

Future Work

Recommendation System

- AI-powered personalized dietary guidance based on detected food and user health goals.

Expanded Food Classes

- Scale beyond 30 classes to cover a wider range of regional Indian dishes.